DM587 - Assignment 2

Least squares, validation and transformed features

Total: 100 points.

Deadline: Tuesday 17 November 2026, 11:59 pm CET (Europe/Copenhagen).

Find Assignment 2 in your private SDU GitLab project `dm587-2026 / YOUR_USERNAME-repo`, folder `asg-linreg`. Download a working copy using the separate student Git guide; keep your earlier assignment work intact.

Implement the seven functions in `asg-linreg/linreg.py`. The questions specify mathematical problems and the required outputs. You must decide how to translate these specifications into correct Python and NumPy code. Standard NumPy numerical routines are allowed; construct the required matrices and implement model selection yourself. SciPy is not required.

Question	Points
1. A patterned matrix, fitted coefficients and RMSE	30
2. Polynomial design and validation-based selection	40
3. A prescribed transformed-feature model	30
Total	100

Local checks and grading

The supplied data are examples for checking your implementation locally. From the `asg-linreg` folder, run `python3 check_examples.py` (Windows: `py -3 check_examples.py`). These are basic checks; passing them does not guarantee full marks. The unfinished starter will fail until you implement the functions. Read failures and correct your implementation, leaving the checks unchanged.

Your functions must work for other valid inputs, not only the supplied examples. The grader also tests different input values, array sizes, polynomial degrees and training/validation splits, as applicable to each question. The mathematical rules and required output shapes remain the same. Always use the arguments passed to your functions; do not hard-code the supplied data, sizes or answers.

What your functions must return

- Return the types and shapes specified in each question. Use NumPy arrays with `float64` precision where an array is requested. A vector of shape `(n,)` is different from a column array of shape `(n, 1)`.
- Return your computed results without rounding. Small floating-point differences are accepted.
- Do not modify the input arrays.
- Implement only the seven marked functions in `linreg.py`. Keep their names and arguments unchanged, and leave the supplied data and check files unchanged.

`assignment2_data.py` generates the observations and splits; `check_examples.py` calls your functions for local checks. There are no data files to load or download. Keep each function's docstring first and write your implementation below it, replacing `pass`.

Marking

Your highest score among on-time submissions will count towards your grade. A submission must be successfully received by GitLab before **Wednesday 18 November 2026, 00:00 CET**. Submissions received at or after that cutoff will not be considered for the grade. Eligibility is based on the time GitLab receives your push; grading may finish afterwards.

The questions total **30 + 40 + 30 = 100 points**. Each listed point group is awarded when its numerical checks pass; there is no partial credit within a group.

Question 1 - A patterned system

30 points: matrix construction (10), coefficient vectors (10), RMSE (10).

1(a) Construct the matrix - 10 points

For the principal dataset, use $m = 200$ **and** $p = 12$. Construct the resulting 200×12 **matrix** using

$$t_i = \frac{i + \frac{1}{2}}{m}, \qquad A_{ij} = \cos(\pi j t_i),$$

where $i = 0, \ldots, m-1$ and $j = 0, \ldots, p-1$. The first subscript identifies the row and the second identifies the column. The formula applies to every entry, including column zero.

Implement `patterned_matrix(m, p)`, returning the matrix `A` as a NumPy array of shape `(m, p)`. This part is graded by comparing your matrix entries and shape with the specified matrix. Your function must also work for other positive integer dimensions with $m > p$.

1(b) Solve two systems - 10 points

Using the matrix from 1(a), consider the two systems

$$A\boldsymbol{x} = \boldsymbol{b}^{(1)}, \qquad A\boldsymbol{x} = \boldsymbol{b}^{(2)},$$

where the right-hand sides are generated automatically from

$$b_i^{(1)} = 1.5 + 0.8\cos(2\pi t_i) - 0.4\cos(5\pi t_i),$$
$$b_i^{(2)} = b_i^{(1)} + 0.1\sin(4\pi t_i) + 0.2\cos(19\pi t_i).$$

Solve each system. If there is no exact solution, return the best approximate solution, meaning the coefficient vector that minimises $\|\boldsymbol{b} - A\boldsymbol{x}\|_2$. The columns of A are independent, so the required coefficient vector is unique in both cases.

Implement `solve_system(A, b)`, returning only the **coefficient array** of shape `(p,)`. The supplied checks call this function separately with each right-hand side. Both input vectors have shape `(200,)`. These **10 points** test the coefficients for both systems. Your function must also handle other tall matrices with independent columns. No particular numerical solver is prescribed.

1(c) Measure prediction error - 10 points

Implement `rmse(residuals)` for a nonempty residual vector of shape `(n,)`. Return one floating-point scalar defined by

$$\mathrm{RMSE}(\boldsymbol{r}) = \sqrt{\frac{1}{n}\sum_{i=0}^{n-1} r_i^2}.$$

For each system in 1(b), form the residual vector using

$$\boldsymbol{r} = \boldsymbol{b} - A\widehat{\boldsymbol{x}}.$$

Apply `rmse(residuals)` to this vector. For an exact solution the RMSE is zero mathematically; a tiny floating-point value is accepted. Always compute the RMSE from the residuals rather than forcing small values to zero.

These **10 points** test the scalar RMSE for both systems, not the residual vector as a separate output. The grader supplies independently computed residuals, as well as other fixed residual vectors, so these points do not depend on your implementation of 1(a) or 1(b) being correct.

Question 2 - Select a polynomial

40 points: design matrix (15), selection (10), coefficients (10), validation errors (5).

The dataset has **120 observations** (x_i, y_i).

The supplied **training set T has 96 observations** and the **validation set V has 24**. Use them unchanged. The supplied index arrays `train_idx` and `validation_idx` identify the sets.

2(a) Construct the training design matrix - 15 points

For a polynomial of degree d, use the model

$$p_d(x;\boldsymbol{\theta}) = \sum_{j=0}^{d} \theta_j x^j.$$

Let $x_0, \ldots, x_{n_{\text{train}}-1}$ be the training inputs. Construct the design matrix $A^{(d)}$, whose entry in row i and column j is

$$A^{(d)}_{ij} = x_i^j, \qquad i = 0, \ldots, n_{\text{train}} - 1, \quad j = 0, \ldots, d.$$

Thus row i contains the polynomial features evaluated at training datapoint x_i:

$$A^{(d)}_{i,:} = \begin{bmatrix} 1 & x_i & x_i^2 & \cdots & x_i^d \end{bmatrix}.$$

The columns contain ones, training inputs, their squares, and successive powers up to degree d, in that order. Do not centre or rescale the inputs.

Implement `polynomial_design(x_train, degree)`.

The function must work for **any one-dimensional input array `x_train` and any nonnegative integer `degree`**, including degree zero. The input array always has shape `(n_train,)`, where `n_train` is its number of entries. Return the corresponding design matrix with shape `(n_train, degree+1)`. Do not hard-code the number of observations or the degree. Apply the same feature definition to validation inputs in 2(b).

2(b) Validation and a simpler model - 25 points

Consider exactly the candidate degrees `DEGREES=(1, 2, 3, 5, 7, 9)`. For each degree, determine the coefficients and validation error defined by

$$\widehat{\boldsymbol{\theta}}^{(d)} = \operatorname*{arg\,min}_{\boldsymbol{\theta}\in\mathbb{R}^{d+1}} \sum_{i\in T} \big(y_i - p_d(x_i;\boldsymbol{\theta})\big)^2,$$

$$E_d = \sqrt{\frac{1}{|V|} \sum_{i\in V} \big(y_i - p_d(x_i;\widehat{\boldsymbol{\theta}}^{(d)})\big)^2}.$$

Validation observations must not influence the fitted coefficients. Let `E_min` be the smallest validation error among the candidate degrees. Select

$$d^* = \min\big\{d \in \{1,2,3,5,7,9\} : E_d \le E_{\min} + 0.0025\big\}.$$

Thus a smaller degree is preferred when its validation RMSE lies within the specified absolute margin of the minimum. Do not change the margin, round errors before selection, or replace this rule with another definition of a tie.

Implement `select_polynomial(x, y, train_idx, validation_idx)`.

Return `(selected_degree, coefficients, validation_errors)`. The selected degree is an integer; its coefficients have shape `(selected_degree+1,)`; the six errors have shape `(6,)`, in `DEGREES` order.

Return the selected model's **training-set coefficients**, without refitting on training plus validation. Selection earns 10 points, selected coefficients 10, and validation errors 5. Use the supplied index arrays, including when the grader supplies another valid split.

Question 3 - Unfamiliar features

30 points: feature matrix (10), coefficients (10), predictions (10).

For an input $x \in [0, 1]$, consider the following prescribed feature functions:

$$\phi_0(x) = 1, \qquad \phi_1(x) = x,$$
$$\phi_2(x) = \max\{0, x - 0.4\},$$
$$\phi_3(x) = \exp\left(-\frac{(x - 0.7)^2}{2(0.15)^2}\right),$$
$$\phi_4(x) = \sin(3\pi x).$$

The constants are part of the specification, not parameters to estimate. In particular, the maximum in the third feature is evaluated separately for each input value.

3(a) Construct the feature matrix - 10 points

Implement `feature_design(x)`. For input shape `(n,)`, return shape `(n, 5)`, with entries $A_{ij} = \phi_j(x_i)$ in the feature order above. Decide how to evaluate the features and arrange the resulting array. Do not add another constant feature, standardise the columns or transform the response.

3(b) Fit coefficients and predict - 20 points

For training observations (x_i, y_i), define

$$f(x; \boldsymbol{\theta}) = \sum_{j=0}^{4} \phi_j(x)\theta_j,$$
$$\widehat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta} \in \mathbb{R}^5} \sum_{i=0}^{n-1} \big(y_i - f(x_i; \boldsymbol{\theta})\big)^2.$$

Implement `fit_features(x_train, y_train, x_eval)`, returning `(coefficients, predictions)`. Coefficients have shape `(5,)` in the prescribed feature order. Predictions have shape `(len(x_eval),)` and are the values of `f` at the evaluation inputs using the fitted coefficients.

The supplied Python code generates and passes `x_train`, `y_train` and `x_eval` automatically: **160 training observations** and **60 evaluation inputs**. The training matrix has independent columns. Fit using training observations only. Your functions must also handle other valid input lengths. Coefficients and predictions earn 10 points each.

Submit Assignment 2 and read your grade

From the local project folder containing `asg-linreg`, save your code and run:

```
cd asg-linreg
python3 check_examples.py
cd ..
git add asg-linreg/linreg.py
git commit -m "Submit Assignment 2"
git push
```

On Windows, replace `python3` with `py -3`. Only a successful push submits your committed file and starts grading. To resubmit, save your changes and repeat these commands.

In GitLab, open **Build > Pipelines**, select your latest commit's pipeline, then open `asg_assignment2`. Read `Assignment 2: Grade: .../100.00` and the failing test names in the log. A Failed job can still have a valid partial score. Feedback is also saved in the job artifacts at `feedback/assignment2/summary.txt`. Contact the instructor if the log says **Score unavailable**.